



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y
MATEMÁTICAS

METAHEURÍSTICAS

Trabajo final:
Sine Cosine Algorithm (SCA)

Autor: Jesús Muñoz Velasco

Curso 2025-2026

Índice

I	Descripción	3
1.	Introducción y contexto	3
2.	Fundamentos matemáticos	3
2.1.	Ecuaciones de actualización de posición	3
2.2.	Papel de cada parámetro	4
2.2.1.	El parámetro r_1 : transición exploración/explotación	4
2.2.2.	El parámetro r_2 : dirección del movimiento	5
2.2.3.	Los parámetros r_3 y r_4	5
2.3.	Geometría de la búsqueda	6
3.	Descripción del algoritmo	7
3.1.	Pseudocódigo	7
3.2.	Complejidad computacional	7
II	Implementación y estudio experimental	7
4.	Implementación del SCA	8
5.	Comparativa experimental: SCA frente a DE y PSO	8
5.1.	Dimensión $d = 10$	9
5.1.1.	Resultados al 100 % de evaluaciones	9
5.1.2.	Análisis de la convergencia a lo largo de las evaluaciones	10
5.2.	Análisis por tipo de función ($d = 10$)	11
5.3.	Resumen y motivación de las mejoras ($d = 10$)	11
5.4.	Dimensión $d = 30$	12
5.4.1.	Resultados al 100 % de evaluaciones	12
5.4.2.	Análisis de la convergencia a lo largo de las evaluaciones	14
5.4.3.	Análisis por tipo de función ($d = 30$)	14
5.5.	Dimensión $d = 50$	15
5.5.1.	Resultados al 100 % de evaluaciones	15
5.5.2.	Análisis de la convergencia a lo largo de las evaluaciones	17
5.6.	Análisis comparativo entre dimensiones	18
III	Hibridación memética: SCA + Solis-Wets (SCA-LS)	18
6.	Motivación	19
7.	Descripción del SCA-LS	19
7.1.	Elección del intensificador: Solis-Wets	19
7.2.	Mecanismo de activación: detección de estancamiento	19
7.3.	Pseudocódigo	20

7.4. Parámetros del SCA-LS	20
8. Estudio experimental del SCA-LS	21
8.1. Dimensión $d = 10$	21
8.1.1. Resultados al 100 % de evaluaciones	21
8.1.2. Análisis de la convergencia a lo largo de las evaluaciones	22
8.2. Dimensión $d = 30$	23
8.2.1. Resultados al 100 % de evaluaciones	23
8.2.2. Análisis de la convergencia a lo largo de las evaluaciones	25
8.3. Dimensión $d = 50$	25
8.3.1. Resultados al 100 % de evaluaciones	25
8.4. Análisis comparativo entre dimensiones	27
8.5. Conclusiones sobre el SCA-LS	27
 IV Mejoras al diseño del SCA: ASCA	 28
9. Motivación y diseño de las mejoras	28
9.1. Mejora 1: Decaimiento no lineal de r_1 (coseno cuadrático)	28
9.1.1. Limitación encontrada	28
9.1.2. Solución propuesta	29
9.2. Mejora 2: Topología de vecindario en anillo (<i>ring lbest</i>)	29
9.2.1. Limitación encontrada	29
9.2.2. Solución propuesta	29
9.3. Mejora 3: Reinicio parcial adaptativo basado en diversidad	30
9.3.1. Limitación encontrada	30
9.3.2. Solución propuesta	30
9.4. Pseudocódigo del ASCA	30
9.5. Parámetros del ASCA	31
 10. Estudio experimental del ASCA	 31
10.1. Dimensión $d = 10$	31
10.1.1. Resultados al 100 % de evaluaciones	31
10.1.2. Análisis de la convergencia a lo largo de las evaluaciones	32
10.2. Dimensión $d = 30$	32
10.3. Dimensión $d = 50$	33
10.4. Análisis crítico: por qué el ASCA no supera al SCA en este benchmark	33
10.5. Comparativa final de los tres algoritmos SCA	34
10.6. Conclusiones generales	35
 Referencias	 36

Parte I

Descripción

1. Introducción y contexto

El Sine Cosine Algorithm (SCA) es una metaheurística poblacional propuesta por Seyedali Mirjalili en 2016 en el artículo “*SCA: A Sine Cosine Algorithm for solving optimization problems*” (Knowledge-Based Systems, vol. 96, pp. 120–133). Su propuesta se enmarca dentro de la familia de algoritmos basados en modelos matemáticos, también denominados *math-inspired*, en contraposición a los inspirados en fenómenos biológicos o físicos.

El SCA parte de la idea de que las funciones seno y coseno poseen propiedades geométricas naturalmente útiles para la búsqueda: oscilan de forma periódica, pueden tomar valores superiores o inferiores al rango $[-1, 1]$ si se escalan convenientemente, y su comportamiento alternante facilita tanto la exploración global del espacio de búsqueda como la explotación local alrededor de soluciones prometedoras. Desde su publicación, el algoritmo ha atraído una atención considerable en la comunidad investigadora, siendo aplicado en campos tan diversos como la ingeniería eléctrica, el aprendizaje automático, el diagnóstico médico y la planificación de rutas.

2. Fundamentos matemáticos

2.1. Ecuaciones de actualización de posición

Cada candidato a solución (denominado *agente de búsqueda*) se representa como un vector $\mathbf{X}^t \in \mathbb{R}^d$, donde d es la dimensión del problema y t el instante de iteración. En cada paso, el agente actualiza su posición dimensión a dimensión según:

$$X_i^{t+1} = \begin{cases} X_i^t + r_1 \cdot \sin(r_2) \cdot |r_3 P_i^t - X_i^t|, & \text{si } r_4 < 0,5 \\ X_i^t + r_1 \cdot \cos(r_2) \cdot |r_3 P_i^t - X_i^t|, & \text{si } r_4 \geq 0,5 \end{cases} \quad (1)$$

La ecuación puede factorizarse para visualizarla mejor. Definiendo el *desplazamiento escalado* $\Delta_i^t = r_3 P_i^t - X_i^t$ (distancia ponderada al destino, con signo) y el *factor trigonométrico* $\phi = r_1 \cdot f(r_2)$ donde $f \in \{\sin, \cos\}$, la actualización se reduce a:

$$X_i^{t+1} = X_i^t + \phi \cdot |\Delta_i^t| \quad (2)$$

El valor absoluto $|\Delta_i^t|$ garantiza que la magnitud del paso sea siempre positiva; el signo del movimiento queda determinado enteramente por ϕ : si $\phi > 0$ el agente avanza hacia el destino (o más allá), si $\phi < 0$ retrocede. Esta separación entre magnitud y dirección es lo que confiere al SCA su capacidad para cubrir simétricamente el espacio alrededor del punto destino.

Los cuatro parámetros tienen roles bien diferenciados:

Param.	Rango	Rol en la actualización
r_1	$[0, a]$ decreciente	Amplitud del movimiento (exploración/explotación)
r_2	$[0, 2\pi]$ aleatorio	Ángulo trigonométrico (dirección y signo del paso)
r_3	$[0, 2]$ aleatorio	Peso estocástico de la distancia al destino
r_4	$[0, 1]$ aleatorio	Selección equiprobable de seno o coseno

2.2. Papel de cada parámetro

2.2.1. El parámetro r_1 : transición exploración/explotación

r_1 es el parámetro de mayor importancia. Viene dado por¹

$$r_1(t) = a \left(1 - \frac{t}{T} \right), \quad a = 2 \quad (3)$$

Como se puede ver comienza con el valor a ($r_1(0) = a$) y va descendiendo linealmente hasta 0 ($r_1(T) = 0$) a lo largo de las iteraciones.

La Figura 1 muestra su trayectoria y el umbral $r_1 = 1$ que separa las dos regiones:

- **Exploración** ($r_1 \geq 1$, primeras iteraciones): el factor $r_1 \cdot f(r_2)$ puede superar en módulo la distancia $|\Delta_i^t|$, de modo que el agente sobrepasa el punto destino y explora regiones lejanas del espacio.
- **Explotación** ($r_1 < 1$): el factor queda acotado y el agente se mueve dentro del segmento que lo une con el destino, refinando la solución progresivamente.

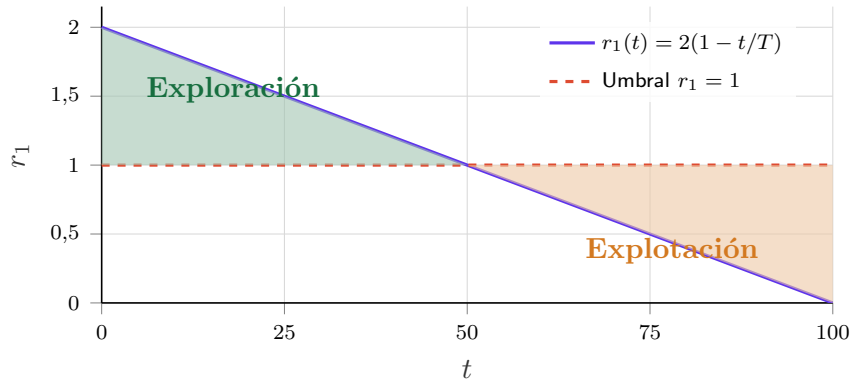


Figura 1: Evolución de r_1 a lo largo de las iteraciones ($a = 2$, $T = 100$). La región azul corresponde a la fase de exploración ($r_1 \geq 1$) y la naranja a la de explotación ($r_1 < 1$). El cruce ocurre exactamente en $t = T/2$.

¹El parámetro a se toma igual a 2 para que el espacio de búsqueda alrededor del punto destino quede completamente cubierto desde el principio (está acotado por el doble de la distancia inicial), además de que con este valor, el umbral $r_1 = 1$ se alcanza exactamente en $t = T/2$ dividiendo las iteraciones perfectamente por la mitad para exploración y explotación. Aún así se puede tomar otro valor para darle más relevancia a alguna de estas fases.

2.2.2. El parámetro r_2 : dirección del movimiento

r_2 es el ángulo de la función trigonométrica, muestreado uniformemente en $[0, 2\pi]$. Su efecto se aprecia analizando el signo de $\sin(r_2)$ y $\cos(r_2)$:

-) Cuando $f(r_2) > 0$ el desplazamiento es positivo: el agente se mueve en la dirección de $P_i - X_i$ (hacia el destino o más allá).
-) Cuando $f(r_2) < 0$ el desplazamiento es negativo: el agente se aleja del destino en esa dimensión.

La Figura 2 representa $r_1 \cdot \sin(r_2)$ y $r_1 \cdot \cos(r_2)$ para dos valores representativos de r_1 , mostrando cómo la amplitud se contrae al pasar de la fase exploratoria a la explotadora.

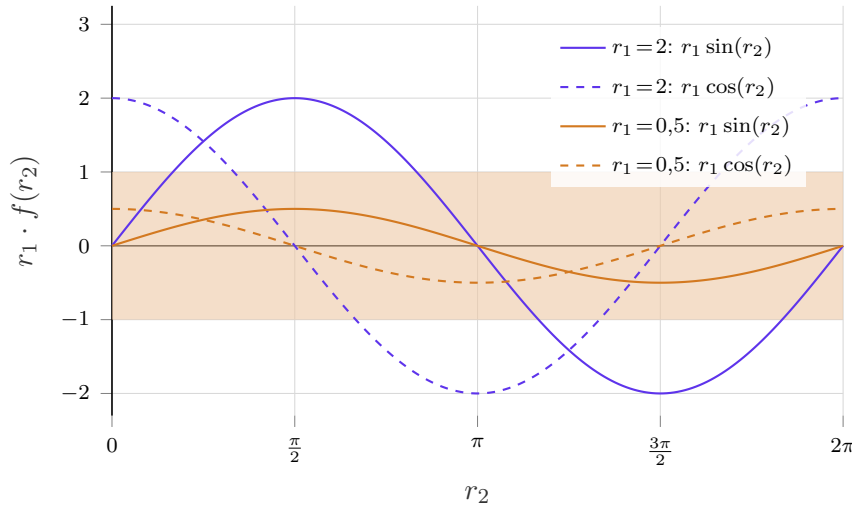


Figura 2: Factor trigonométrico $r_1 \cdot f(r_2)$ en función del ángulo r_2 para $r_1 = 2$ (exploración, azul) y $r_1 = 0,5$ (explotación, naranja). La banda sombreada $[-1, 1]$ delimita el régimen de explotación: cuando el factor queda dentro de ella, el agente no sobrepasa el destino.

2.2.3. Los parámetros r_3 y r_4

$r_3 \in [0, 2]$ actúa como un peso estocástico sobre el punto destino. Si $r_3 > 1$, el término $r_3 P_i^t$ desplaza el destino efectivo más allá de su posición real, ampliando artificialmente la distancia $|\Delta_i^t|$ y favoreciendo pasos más largos (efecto exploratorio secundario). Si $r_3 < 1$, atenúa la influencia del destino. Este mecanismo introduce diversidad adicional incluso en la fase de explotación, reduciendo el riesgo de convergencia en masa hacia un único punto.

$r_4 \in [0, 1]$ selecciona con probabilidad $1/2$ entre la función seno y la función coseno. Ambas funciones tienen la misma amplitud máxima, pero sus ceros y máximos están desfasados $\pi/2$ radianes. Emplearlas con igual probabilidad garantiza que el muestreo de direcciones de movimiento sea más uniforme que si se usase una sola función, mejorando la isotropía de la búsqueda en el espacio.

2.3. Geometría de la búsqueda

La ecuación (2) actualiza cada dimensión de forma independiente. En una dimensión fija j , el nuevo valor del agente es:

$$X_j^{t+1} = X_j^t + \phi \cdot |\Delta_j^t|, \quad |\Delta_j^t| = |r_3 P_j^t - X_j^t|, \quad \phi = r_1 \cdot f(r_2)$$

donde $|\Delta_j^t| \geq 0$ siempre. Por tanto, el signo y la magnitud del desplazamiento quedan determinados exclusivamente por ϕ , que puede tomar cualquier valor real en $[-r_1, r_1]$. Esto da lugar a cuatro regímenes de movimiento, representados en la Figura 3:

- $\phi \in (0, 1)$: explotación positiva. El agente se desplaza hacia P_j^t sin alcanzarlo.
- $\phi \in (-1, 0)$: explotación negativa. El agente retrocede levemente desde P_j^t , pero permanece cerca.
- $\phi > 1$: exploración positiva. El agente sobrepasa P_j^t y aterriza más allá.
- $\phi < -1$: exploración negativa. El agente da un salto largo en sentido opuesto a P_j^t .

Como ya se ha comentado, los casos $|\phi| < 1$ constituyen la zona de explotación y los casos $|\phi| > 1$ la zona de exploración. El umbral es exactamente $|\phi| = 1$, que corresponde al punto P_j^t escalado por r_3 . Nótese que las zonas de exploración son simétricas a ambos lados del agente, lo que garantiza que la búsqueda no tenga sesgo de dirección.

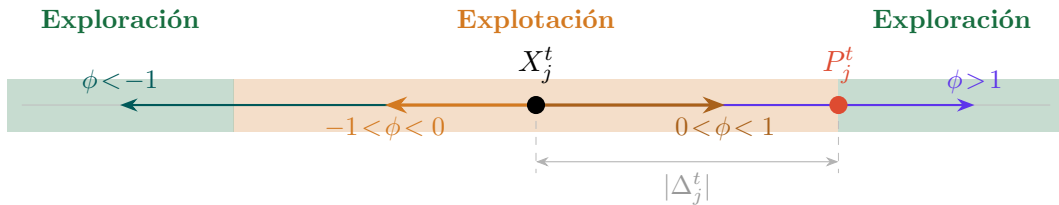


Figura 3: Cuatro regímenes de movimiento en una dimensión j . La banda amarilla ($|\phi| < 1$) es la zona de explotación: el agente aterriza entre X_j^t y el destino (o su simétrico). Las bandas azul y verde ($|\phi| > 1$) son zonas de exploración: el agente sobrepasa el destino en alguno de los dos sentidos. La simetría horizontal garantiza la ausencia de sesgo direccional.

Desde una perspectiva probabilística, el valor esperado de $|\phi|$ decrece con t al hacerlo r_1 , ya que

$$E[|\phi|] = r_1 \cdot E[|f(r_2)|] = r_1 \cdot \frac{2}{\pi}$$

Esto convierte la dinámica colectiva en un proceso de enfriamiento implícito: al inicio los agentes realizan saltos amplios y divergentes conforme avanza la búsqueda los pasos se acortan y la población converge hacia el punto destino con precisión creciente. El factor $2/\pi \approx 0,637$ implica además que, en promedio, el umbral efectivo de exploración/explotación se alcanza antes de que r_1 llegue a 1, lo que supone una transición algo más temprana de lo que la ecuación (3) sugiere a simple vista.

3. Descripción del algoritmo

3.1. Pseudocódigo

```

1  (* Parámetros de entrada: N (población), T (iteraciones máx.), a=2 *)
2  Inicializar población X = {X_1, X_2, ..., X_N} aleatoriamente en [lb, ub]^d
3  Evaluar fitness f(X_i) para cada agente i
4  P <- argmin f(X_i)      (* mejor solución: punto destino *)
5
6  t <- 1
7  Mientras t <= T hacer
8      (* Calcular parámetro adaptativo r1 *)
9      r1 <- a - t * (a / T)
10
11     Para cada agente i = 1 hasta N hacer
12         Para cada dimensión j = 1 hasta d hacer
13             (* Generar parámetros aleatorios *)
14             r2 <- aleatorio en [0, 2*pi]
15             r3 <- aleatorio en [0, 2]
16             r4 <- aleatorio en [0, 1]
17
18             (* Actualizar posición según Ec. (1) *)
19             Si r4 < 0.5 entonces
20                 X_i[j] <- X_i[j] + r1 * sin(r2) * |r3*P[j] - X_i[j]|
21             Si_no
22                 X_i[j] <- X_i[j] + r1 * cos(r2) * |r3*P[j] - X_i[j]|
23             Fin Si
24
25             (* Aplicar límites del espacio de búsqueda *)
26             X_i[j] <- max(lb[j], min(ub[j], X_i[j]))
27         Fin Para
28     Fin Para
29
30     (* Evaluar fitness y actualizar punto destino *)
31     Para cada agente i = 1 hasta N hacer
32         Evaluar f(X_i)
33         Si f(X_i) < f(P) entonces
34             P <- X_i
35         Fin Si
36     Fin Para
37
38     t <- t + 1
39 Fin Mientras
40
41 Retornar P      (* mejor solución encontrada *)

```

Código Fuente 1: Pseudocódigo del Sine Cosine Algorithm (SCA)

3.2. Complejidad computacional

La complejidad por iteración es $\mathcal{O}(N \cdot d)$, equivalente a la de otros algoritmos poblacionales estándar como PSO o GWO. El número total de evaluaciones de la función objetivo es $N \times T$, lo que resulta razonable para problemas de mediana escala.

Parte II

Implementación y estudio experimental

4. Implementación del SCA

La implementación del SCA se ha realizado en C++ sobre el *benchmark* CEC2017 [7], integrándola en el repositorio de Daniel Molina [8]. La API de evaluación ofrece una interfaz simplificada que gestiona internamente el conteo de evaluaciones y el almacenamiento de resultados en los milestones requeridos por el protocolo experimental.

Los parámetros de la implementación se recogen en la Tabla 1. Se ha optado por $N = 30$ agentes, que es el valor habitual que se suele usar para CEC2017 para mejorar la comparativa con los otros algoritmos. El criterio de parada establecido es $10\,000 \times d$ evaluaciones y el número de repeticiones se ha fijado a $T = 10$ (frente al estándar de 50) para acotar los tiempos de ejecución.

Parámetro	Valor	Descripción
N	30	Tamaño de población
a	2,0	Amplitud inicial de r_1
$T_{\text{máx}}$	$10\,000 \times d$	Presupuesto máximo de evaluaciones
T	10	Repeticiones por función
Rango	$[-100, 100]^d$	Dominio de búsqueda
Dimensiones	10, 30, 50	Valores de d evaluados

Tabla 1: Parámetros de la implementación del SCA para CEC2017.

5. Comparativa experimental: SCA frente a DE y PSO

En esta sección se analiza el comportamiento del SCA frente a dos algoritmos clásicos de referencia: la *Differential Evolution* (DE) y el *Particle Swarm Optimization* (PSO). Ambos están disponibles en la base de datos de TacoLab [9] con el mismo protocolo experimental. La comparativa se realiza sobre las 30 funciones del benchmark CEC2017 en tres dimensiones ($d \in \{10, 30, 50\}$), midiendo el error medio respecto al óptimo conocido en distintos milestones del presupuesto de evaluaciones ($10\,000 \times d$ en total).

5.1. Dimensión $d = 10$

5.1.1. Resultados al 100 % de evaluaciones

La Tabla 2 muestra los errores medios al usar el 100 % de las evaluaciones disponibles (10^5 evaluaciones para $d = 10$). Las celdas corresponden al error medio sobre 10 ejecuciones (cuanto menor sea dicho valor mejor será el algoritmo).

Tabla 2: Error medio al 100 % de evaluaciones ($d = 10$, 10^5 evaluaciones). Mejor resultado por función destacado en negrita.

Función	DE	PSO	SCA
F01	0.000e+00	5.255e+07	5.431e+08
F02	0.000e+00	1.000e+00	9.754e+05
F03	0.000e+00	1.989e+03	6.766e+02
F04	1.105e-04	4.684e+01	2.693e+01
F05	1.151e+02	3.212e+01	4.856e+01
F06	3.460e+01	1.001e+01	1.405e+01
F07	3.848e+01	4.275e+01	6.702e+01
F08	2.983e+01	2.203e+01	3.716e+01
F09	1.938e+02	5.686e+01	8.651e+01
F10	3.597e+02	1.077e+03	1.197e+03
F11	1.942e-02	3.843e+01	7.486e+01
F12	4.931e+00	2.517e+06	7.962e+06
F13	5.988e+00	8.409e+03	2.355e+04
F14	5.240e-02	9.993e+01	1.619e+02
F15	6.060e-02	2.066e+03	5.189e+02
F16	4.561e+02	1.415e+02	1.448e+02
F17	2.350e+01	6.497e+01	6.609e+01
F18	3.630e-02	1.484e+04	7.227e+04
F19	5.192e-03	3.220e+03	5.461e+02
F20	3.837e+02	8.444e+01	9.014e+01
F21	1.889e+02	1.320e+02	1.509e+02
F22	1.005e+02	7.740e+01	1.512e+02
F23	8.098e+02	3.304e+02	3.513e+02
F24	1.000e+02	1.810e+02	3.824e+02
F25	4.040e+02	4.478e+02	4.537e+02
F26	2.706e+02	3.729e+02	4.519e+02
F27	3.897e+02	4.134e+02	4.010e+02
F28	3.517e+02	4.698e+02	4.734e+02
F29	2.375e+02	3.193e+02	3.363e+02
F30	8.051e+04	6.352e+05	4.864e+05
Mejor en	21	9	0

El DE obtiene el mejor resultado en 21 de las 30 funciones, el PSO en 9, y el SCA en ninguna. Este resultado, aunque no beneficia a SCA, es coherente con las limitaciones teóricas identificadas en la descripción (Parte I): el decrecimiento lineal

de r_1 activa la explotación demasiado pronto, y la ausencia de memoria colectiva entre agentes dificulta la convergencia en funciones multimodales.

Conviene destacar, sin embargo, que en F03, F04, F19 y F27 el SCA se aproxima al rendimiento del PSO, lo que sugiere que en funciones con estructura geométrica favorable a los movimientos trigonométricos el algoritmo se defiende razonablemente.

5.1.2. Análisis de la convergencia a lo largo de las evaluaciones

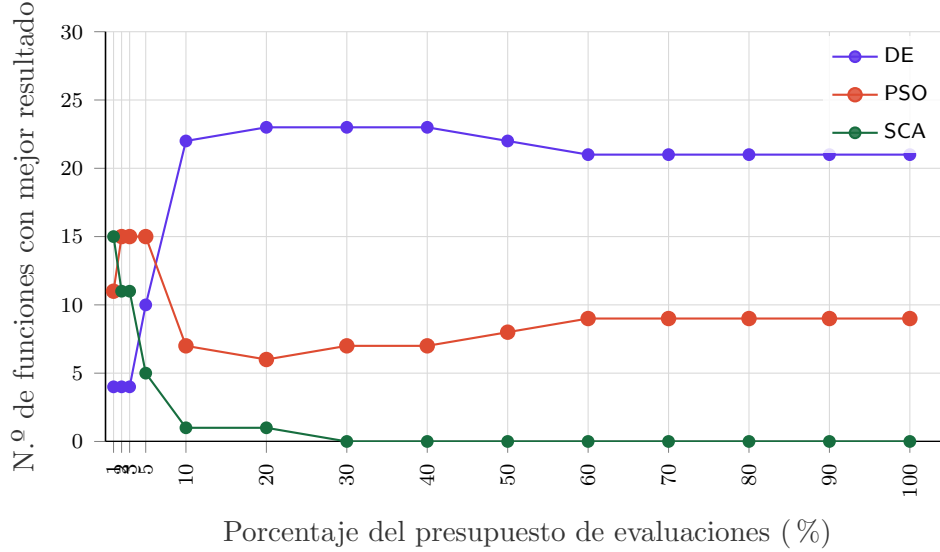


Figura 4: Evolución del número de funciones para las que cada algoritmo resulta mejor en función del porcentaje de las evaluaciones consumidas ($d = 10, 30$ funciones).

La Figura 4 revela una dinámica especialmente interesante que los resultados finales por sí solos ocultan. Al 1 % de las evaluaciones (los primeros 10^3 de los 10^5 evaluaciones disponibles), el SCA gana en 15 de las 30 funciones, superando tanto al DE (4 victorias) como al PSO (11 victorias). Esta ventaja inicial refleja la alta capacidad exploratoria del SCA en las primeras iteraciones, cuando r_1 es próximo a $a = 2$ y los agentes se dispersan ampliamente por el espacio de búsqueda.

Sin embargo, a partir del 5 % de las evaluaciones la situación se invierte: el SCA pierde todas sus ventajas y al 30 % ya no gana en ninguna función. Este colapso es la manifestación directa de las limitaciones ya identificadas: el decrecimiento lineal de r_1 reduce la amplitud de movimiento demasiado rápido, y sin un mecanismo de escape local, la población converge de forma prematura a óptimos locales (impidiéndole alcanzar óptimos globales).

El DE, por el contrario, muestra un comportamiento opuesto: al principio cuenta con pocas victorias (4 al 1 %) y va mejorando progresivamente hasta estabilizarse en torno a 21-23 victorias. Esto refleja su estrategia de intensificación gradual, basada en diferencias entre vectores de la población que son más informativas conforme los agentes convergen. El PSO ocupa una posición intermedia, con buena exploración

inicial (11-15 victorias entre el 1 % y el 5 %) y una posición estable de 7-9 victorias al final.

5.2. Análisis por tipo de función ($d = 10$)

El benchmark CEC2017 agrupa las 30 funciones en tres categorías que permiten caracterizar el comportamiento de cada algoritmo con mayor granularidad:

-) **F01–F03: Funciones unimodales.** El DE domina claramente, alcanzando errores nulos o casi nulos al final del presupuesto. El SCA obtiene resultados razonables en F03 ($6,77 \times 10^2$ vs $1,99 \times 10^3$ del PSO), pero no puede competir con DE en F01 y F02. En F01 (esfera desplazada), el error del SCA ($5,43 \times 10^8$) es 8 órdenes de magnitud mayor que el de DE, lo que confirma que su mecanismo de explotación es insuficiente para funciones convexas.
-) **F04–F10: Funciones multimodales simples.** Es el grupo donde el SCA se comporta de forma más competitiva relativamente. En F05, F06 y F09 obtiene resultados cercanos al PSO, aunque sin superarlo. Destaca F09 (función de Schwefel modificada), donde el SCA ($8,65 \times 10^1$) supera al DE ($1,94 \times 10^2$), lo que sugiere que la oscilación sinusoidal es útil para navegar paisajes con muchos óptimos locales de magnitud comparable.
-) **F11–F20: Funciones híbridas.** El SCA muestra sus peores resultados en este grupo al compararlos con los otros dos algoritmos. Las funciones híbridas combinan regiones con características muy dispares, lo que requiere tanto exploración global como explotación local precisa. La ausencia de búsqueda local en el SCA hace que los agentes no puedan refinar las soluciones parciales encontradas en cada subcomponente. F12 y F18 son los casos más extremos, con errores del SCA entre 3 y 4 órdenes de magnitud superiores al DE.
-) **F21–F30: Funciones de composición.** Son las que presentan el paisaje de aptitud más complejo, con múltiples óptimos globales de distinta naturaleza. Aquí el SCA obtiene sus mejores resultados relativos de entre todas las categorías: en F27 ($4,01 \times 10^2$) supera al PSO ($4,13 \times 10^2$), y en F23, F29 se acerca a los valores del PSO. La diversidad mantenida por la oscilación trigonométrica ayuda a no quedar atrapado en la peor cuenca local.

5.3. Resumen y motivación de las mejoras ($d = 10$)

La Tabla 3 condensa la evolución de las victorias a lo largo del presupuesto para los tres algoritmos.

Tabla 3: Número de funciones ganadas por algoritmo en hitos representativos ($d = 10$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	4	10	22	23	22	21
PSO	11	15	7	6	8	9
SCA	15	5	1	1	0	0

Del análisis se extraen tres conclusiones que motivan directamente las propuestas de mejora desarrolladas en las secciones siguientes:

1. **El SCA explora bien pero no explota.** La ventaja exploratoria inicial (15 victorias al 1 %) se pierde completamente en la fase de explotación, lo que apunta a que el decaimiento lineal de r_1 es demasiado agresivo para funciones multimodales y a que el algoritmo carece de un mecanismo de intensificación local.
2. **La convergencia prematura es el problema principal.** A partir del 10 % del presupuesto el SCA prácticamente no mejora sus posiciones relativas, indicando que la población colapsa hacia óptimos locales antes de haber agotado el presupuesto disponible.
3. **La escasa memoria colectiva limita la cooperación.** Todos los agentes convergen al mismo punto destino P , lo que reduce la diversidad más rápido de lo deseable. Una topología de vecindario permitiría mantener múltiples frentes de búsqueda semi-independientes.

Estas tres debilidades están directamente asociadas a las tres mejoras implementadas en el ASCA (Sección 9): decaimiento no lineal de r_1 , topología de vecindario en anillo, y reinicio parcial adaptativo basado en diversidad.

5.4. Dimensión $d = 30$

Con el salto a $d = 30$, el número disponible de evaluaciones asciende a 3×10^5 , pero el espacio de búsqueda crece exponencialmente. Este cambio de escala tiene un impacto cualitativo importante en el comportamiento relativo de los tres algoritmos.

5.4.1. Resultados al 100 % de evaluaciones

Tabla 4: Error medio al 100 % del presupuesto de evaluaciones ($d = 30$, 3×10^5 evaluaciones). Mejor resultado por función destacado en negrita.

Función	DE	PSO	SCA
F01	4.909e+04	4.175e+09	1.203e+10
F02	1.309e+19	1.000e+00	1.019e+33
F03	3.481e+03	5.453e+04	3.329e+04

Tabla 4 (continuación)

Función	DE	PSO	SCA
F04	8.430e+01	1.183e+03	9.326e+02
F05	2.015e+02	2.170e+02	2.775e+02
F06	6.320e+00	3.694e+01	4.951e+01
F07	2.334e+02	3.596e+02	4.457e+02
F08	1.894e+02	1.745e+02	2.465e+02
F09	6.530e+01	2.842e+03	4.804e+03
F10	3.764e+03	6.938e+03	7.126e+03
F11	7.958e+01	1.207e+03	1.094e+03
F12	3.258e+05	3.586e+08	1.130e+09
F13	1.536e+02	4.508e+07	4.346e+08
F14	7.100e+01	3.059e+05	1.187e+05
F15	6.256e+01	2.737e+05	1.389e+07
F16	1.319e+03	1.568e+03	2.024e+03
F17	4.809e+02	4.725e+02	7.155e+02
F18	6.122e+01	2.170e+06	2.743e+06
F19	3.572e+01	1.260e+06	2.300e+07
F20	2.751e+02	4.621e+02	5.654e+02
F21	3.255e+02	4.113e+02	4.480e+02
F22	1.002e+02	1.027e+03	5.436e+03
F23	5.346e+02	6.404e+02	7.062e+02
F24	6.059e+02	7.095e+02	7.698e+02
F25	3.870e+02	6.864e+02	6.815e+02
F26	4.038e+02	3.369e+03	4.550e+03
F27	4.925e+02	8.068e+02	6.984e+02
F28	3.943e+02	1.105e+03	1.037e+03
F29	1.025e+03	1.409e+03	1.734e+03
F30	3.657e+03	1.359e+07	7.415e+07
Mejor en	27	3	0

Se puede ver que para $d = 30$, la brecha con respecto a $d = 10$ se amplía notablemente. El DE domina en 27 de las 30 funciones (frente a 21 en $d = 10$), el PSO retiene solo 3 victorias, y el SCA no gana en ninguna. Pese a ello, en F03 el SCA ($3,33 \times 10^4$) se coloca entre el PSO ($5,45 \times 10^4$) y el DE ($3,48 \times 10^3$), y en F27 supera al PSO ($8,07 \times 10^2$ vs. $6,98 \times 10^2$), la única función de composición donde el SCA obtiene la segunda posición. F02 sigue siendo el caso donde el PSO arrasa al DE y al SCA gracias a su mecanismo de inercia, pero para el resto del benchmark el DE consolida su hegemonía con el aumento de dimensión.

5.4.2. Análisis de la convergencia a lo largo de las evaluaciones

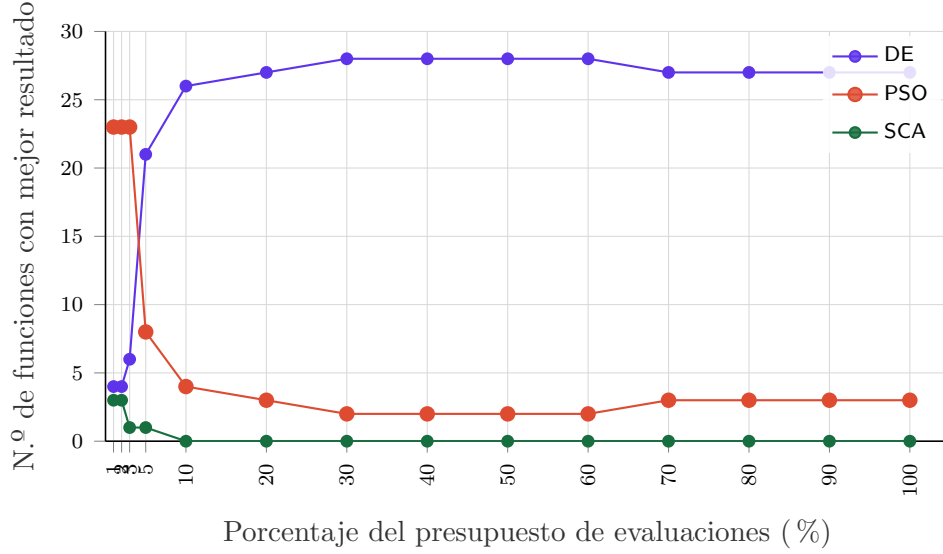


Figura 5: Evolución del número de funciones ganadas por cada algoritmo en función del porcentaje de las evaluaciones consumidas ($d = 30$, 30 funciones).

La Figura 5 muestra un patrón cualitativamente distinto al de $d = 10$. Al 1 % del presupuesto el PSO domina con 23 victorias, el DE obtiene solo 4, y el SCA desciende a 3 victorias, perdiendo la ventaja exploratoria inicial que mostraba en baja dimensión. Este comportamiento refleja que, en $d = 30$, el espacio de búsqueda es tan vasto que la dispersión trigonométrica del SCA resulta insuficiente para cubrir las regiones prometedoras en las primeras evaluaciones.

A partir del 5 % del presupuesto la situación se invierte bruscamente: el DE sube a 21 victorias y el PSO cae a 8, mientras que el SCA ya solo gana en 1 función. Desde el 10 % en adelante el SCA no gana en ninguna, confirmando que la convergencia prematura se adelanta a mayor dimensión. El PSO mantiene una cuota modesta (2–3 victorias) en la fase final gracias a su velocidad de partícula, que le confiere algo de inercia exploratoria residual.

Tabla 5: Número de funciones ganadas por algoritmo en hitos representativos ($d = 30$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	4	21	26	27	28	27
PSO	23	8	4	3	2	3
SCA	3	1	0	0	0	0

5.4.3. Análisis por tipo de función ($d = 30$)

- **F01–F03: Funciones unimodales.** El DE consolida su ventaja en F01 y F09, mientras que el SCA consigue colocarse por delante del PSO en F03. En

F02, el PSO obtiene de nuevo el mejor resultado (1,00) gracias a la naturaleza de la función, que penaliza fuertemente las explosiones de valor características del DE y el SCA en dimensiones altas.

-) **F04–F10: Funciones multimodales simples.** El DE domina en casi todas. El SCA presenta errores entre 3 y 12 veces superiores al DE en F05–F07, y en F09 la brecha se amplía drásticamente respecto a $d = 10$: el SCA comete un error $7,36\times$ mayor que el DE. La oscilación trigonométrica, que en baja dimensión ayudaba a escapar de mínimos locales, no escala bien al aumentar el número de dimensiones activas.
-) **F11–F20: Funciones híbridas.** Es el grupo donde el SCA más regresa en términos relativos. En F15, el error del SCA ($1,39 \times 10^7$) supera al del PSO ($2,74 \times 10^5$) en más de dos órdenes de magnitud, y al del DE ($6,26 \times 10^1$) en cinco órdenes. F13 y F19 presentan brechas similares: el SCA no logra refinar las soluciones parciales porque la convergencia prematura lo atrapa en la parte del paisaje de cada subcomponente que primero encuentra.
-) **F21–F30: Funciones de composición.** El SCA obtiene su único resultado competitivo en F27, donde supera al PSO. En el resto, las brechas respecto al DE son entre 1,5 y 20 veces superiores. La diversidad de cuencas de atracción en estas funciones requeriría múltiples frentes de búsqueda independientes, algo que el SCA con destino global único no puede mantener al crecer la dimensión.

5.5. Dimensión $d = 50$

En $d = 50$ el presupuesto es de 5×10^5 evaluaciones, pero la maldición de la dimensionalidad pesa aún más. Los resultados confirman y amplifican las tendencias observadas en $d = 30$.

5.5.1. Resultados al 100 % de evaluaciones

Tabla 6: Error medio al 100 % del presupuesto de evaluaciones ($d = 50$, 5×10^5 evaluaciones). Mejor resultado por función destacado en negrita.

Función	DE	PSO	SCA
F01	2.039e+08	1.823e+10	4.200e+10
F02	5.158e+52	1.000e+00	4.894e+61
F03	7.597e+04	1.008e+05	1.009e+05
F04	2.322e+02	4.091e+03	6.231e+03
F05	3.978e+02	4.328e+02	5.500e+02
F06	1.180e+01	5.450e+01	6.641e+01
F07	4.781e+02	8.197e+02	9.024e+02
F08	4.070e+02	4.077e+02	5.312e+02
F09	3.756e+03	1.620e+04	2.111e+04
F10	1.127e+04	1.295e+04	1.341e+04
F11	2.173e+02	3.151e+03	4.689e+03
F12	1.382e+08	6.062e+09	1.075e+10

Tabla 6 (continuación)

Función	DE	PSO	SCA
F13	2.726e+04	5.262e+08	2.768e+09
F14	1.683e+02	1.558e+06	1.931e+06
F15	5.090e+02	5.790e+06	3.290e+08
F16	3.047e+03	2.745e+03	3.815e+03
F17	1.827e+03	1.652e+03	2.508e+03
F18	2.420e+04	1.203e+07	1.513e+07
F19	1.322e+02	1.336e+07	1.926e+08
F20	9.384e+02	1.282e+03	1.754e+03
F21	6.177e+02	6.655e+02	7.709e+02
F22	1.707e+02	1.258e+04	1.384e+04
F23	8.435e+02	1.098e+03	1.226e+03
F24	8.850e+02	1.206e+03	1.252e+03
F25	5.667e+02	3.069e+03	3.419e+03
F26	5.745e+02	7.890e+03	9.011e+03
F27	5.765e+02	1.744e+03	1.678e+03
F28	5.013e+02	3.468e+03	3.597e+03
F29	2.218e+03	3.566e+03	4.390e+03
F30	2.386e+06	3.374e+08	4.810e+08
Mejor en	27	3	0

El panorama a $d = 50$ replica el de $d = 30$ en cuanto a recuento de victorias: DE gana en 27 funciones, PSO en 3 (F02, F16, F17) y el SCA en ninguna. Sin embargo, las magnitudes de los errores son cualitativamente distintas. En F15, el SCA alcanza un error de $3,29 \times 10^8$, frente a $5,09 \times 10^2$ del DE —seis órdenes de magnitud de diferencia—. Del mismo modo, en F13 la brecha es de cinco órdenes ($2,77 \times 10^9$ vs. $2,73 \times 10^4$), y en F02 el error del SCA ($4,89 \times 10^{61}$) es literalmente astronómico. Estas cifras reflejan la inestabilidad numérica que surge cuando el decaimiento de r_1 no frena la explosión de valores en funciones mal condicionadas de alta dimensión.

Cabe destacar que a $d = 50$ el PSO amplía ligeramente su posición competitiva en el grupo de funciones multimodales: F16 y F17 pasan a ser victorias del PSO (en $d = 10$ y $d = 30$ eran victorias del DE para F16 o del PSO para F17 respectivamente), lo que sugiere que la memoria de velocidad del PSO escala algo mejor que las diferencias vectoriales del DE en estas funciones específicas.

5.5.2. Análisis de la convergencia a lo largo de las evaluaciones

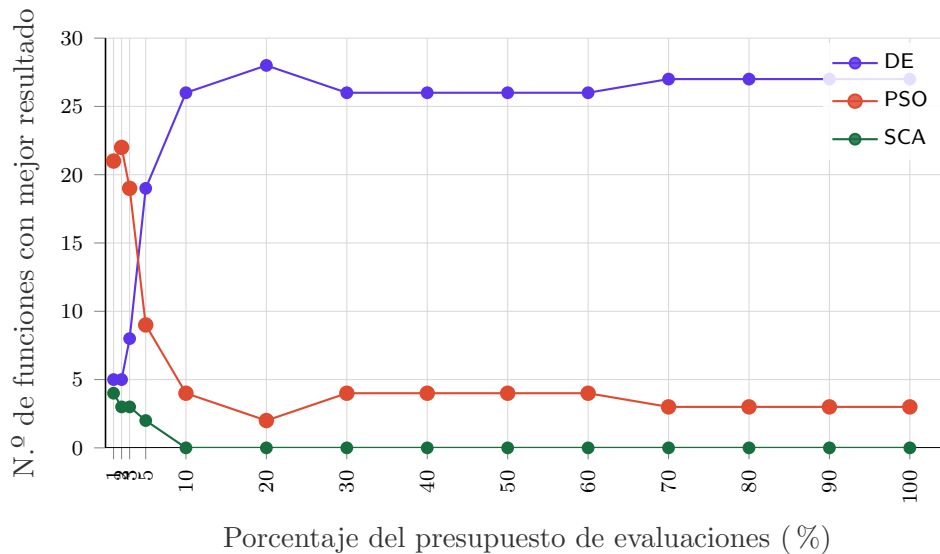


Figura 6: Evolución del número de funciones ganadas por cada algoritmo en función del porcentaje del presupuesto consumido ($d = 50$, 30 funciones).

Tabla 7: Número de funciones ganadas por algoritmo en hitos representativos ($d = 50$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	5	19	26	28	26	27
PSO	21	9	4	2	4	3
SCA	4	2	0	0	0	0

La dinámica de la Figura 6 reproduce a $d = 50$ el mismo patrón cualitativo que a $d = 30$: el PSO lidera al 1 % del presupuesto (21 victorias) gracias a su componente de velocidad que le permite explorar rápidamente en la fase inicial, el DE se impone a partir del 5 % y el SCA desaparece del ranking desde el 10 %. El SCA solo mantiene 2–4 victorias en el primer 5 % del presupuesto, evidenciando que incluso la exploración inicial se ve seriamente comprometida cuando la dimensión triplica la del escenario para el que el algoritmo original fue concebido.

5.6. Análisis comparativo entre dimensiones

Tabla 8: Número de victorias al 100 % del presupuesto por algoritmo y dimensión.

Algoritmo	$d = 10$	$d = 30$	$d = 50$
DE	21	27	27
PSO	9	3	3
SCA	0	0	0

La Tabla 8 condensa el efecto de la dimensión en el balance de victorias finales. Destacan tres tendencias:

1. **El DE escala mejor que PSO y SCA.** El número de victorias del DE crece de 21 a 27 al pasar de $d = 10$ a $d = 30$, y se mantiene en 27 a $d = 50$. Esto indica que la mutación diferencial, basada en diferencias entre vectores de la población, extrae información estadística más útil conforme la población converge en espacios de mayor dimensión.
2. **El PSO sufre una caída severa.** Las 9 victorias a $d = 10$ se reducen a 3 en ambas dimensiones superiores. Aunque la componente de velocidad del PSO permite una exploración inicial rápida (domina el 1 % del presupuesto a $d = 30$ y $d = 50$), su mecanismo de actualización centrado en el mejor global (*gbest*) induce convergencia prematura igual o peor que el SCA en el largo plazo.
3. **El SCA no escala.** Sus 0 victorias son constantes en las tres dimensiones. Si bien a $d = 10$ el SCA mostraba potencial exploratorio real en las primeras iteraciones (15 victorias al 1 %), ese potencial desaparece a $d = 30$ y queda reducido a 4 victorias al 1 % en $d = 50$. La razón es estructural: el SCA actualiza cada dimensión de forma independiente usando el mismo punto destino global, sin ningún mecanismo de adaptación al paisaje de aptitud. En dimensiones altas, la probabilidad de que la actualización trigonométrica simultánea de todas las dimensiones produzca un movimiento útil decrece exponencialmente.

El análisis por hitos (Tablas 3, 5 y 7) revela un patrón consistente: a medida que aumenta d , el SCA pierde la ventaja exploratoria inicial de forma más pronunciada, colapsando desde el 10 % del presupuesto independientemente de la dimensión. Esto refuerza la conclusión de la Sección 5: la mejora prioritaria para el SCA es introducir un mecanismo de memoria colectiva o reinicio de diversidad que prevenga el colapso temprano, especialmente crítico en alta dimensión.

Parte III

Hibridación memética: SCA + Solis-Wets (SCA-LS)

6. Motivación

El análisis experimental del SCA original (Parte II) reveló un patrón claro: el algoritmo exhibe una buena capacidad exploratoria en las primeras iteraciones pero colapsa prematuramente en la fase de explotación, sin llegar a refinar las soluciones encontradas. La conclusión natural es hibridarlo con una búsqueda local que actúe precisamente cuando el SCA se estanca, redistribuyendo así el presupuesto de evaluaciones de forma más eficiente.

La estrategia elegida sigue el paradigma de los *algoritmos meméticos* (MA, *Memetic Algorithms*): una metaheurística poblacional como explorador global combinada con un método de búsqueda local como intensificador. En concreto, se propone el algoritmo **SCA-LS**, que integra el SCA con el método de **Solis-Wets** (SW) [?] como intensificador local activado por estancamiento.

7. Descripción del SCA-LS

7.1. Elección del intensificador: Solis-Wets

El método de Solis-Wets es un optimizador local estocástico sin gradiente diseñado para funciones continuas. Genera perturbaciones gaussianas alrededor de la solución actual, acepta el nuevo punto si mejora el fitness, y ajusta adaptativamente la desviación típica σ en función del historial de éxitos y fracasos. Sus propiedades clave para esta hibridación son:

-) **Sin gradiente**: compatible con todas las funciones del benchmark CEC2017, incluidas las no diferenciables.
-) **Adaptativo**: la escala de perturbación se ajusta automáticamente, evitando la necesidad de calibrar un hiperparámetro de paso.
-) **Ligero**: su implementación consume pocas evaluaciones por llamada, lo que lo hace idóneo para llamadas frecuentes con presupuesto limitado.

7.2. Mecanismo de activación: detección de estancamiento

La búsqueda local se activa cuando el SCA no mejora el mejor global durante $S_{\text{máx}} = 10$ generaciones consecutivas. Este criterio es puramente pasivo: no requiere evaluar ninguna métrica de diversidad adicional y añade un overhead computacional nulo. Al activarse, se asigna a la búsqueda local un presupuesto fijo del 10 % del presupuesto

total por llamada ($\rho = 0,10$), limitado al presupuesto restante para no sobrepasar el criterio de parada.

7.3. Pseudocódigo

```

1  (* Parámetros: N, T_max, a=2, S_max=10, rho=0.10 *)
2  Inicializar población X en [-100,100]^d; evaluar f(X_i)
3  P <- argmin f(X_i); estancamiento <- 0
4
5  Mientras evals < T_max hacer
6      prev_best <- f(P)
7      r1 <- a * (1 - evals / T_max)          (* decaimiento lineal *)
8
9      (* Fase SCA estándar *)
10     Para cada agente i hacer
11         Para cada dimensión j hacer
12             Actualizar X_i[j] con sin/cos (igual que SCA)
13         Fin Para
14         Evaluar f(X_i); actualizar P si mejora
15     Fin Para
16
17     (* Detección de estancamiento *)
18     Si f(P) < prev_best entonces
19         estancamiento <- 0
20     Si_no
21         estancamiento <- estancamiento + 1
22     Fin Si
23
24     (* Activación del intensificador local *)
25     Si estancamiento >= S_max entonces
26         budget_ls <- min(rho * T_max, T_max - evals)
27         ls_sol, ls_evals <- Solis_Wets(P, budget_ls)
28         evals <- evals + ls_evals
29         Si f(ls_sol) < f(P) entonces P <- ls_sol Fin Si
30         estancamiento <- 0
31     Fin Si
32 Fin Mientras
33
34 Retornar P

```

Código Fuente 2: Pseudocódigo del SCA-LS (SCA + Solis-Wets)

7.4. Parámetros del SCA-LS

Parámetro	Valor	Descripción
N	30	Tamaño de población
a	2,0	Amplitud inicial de r_1 (igual que SCA)
$S_{\text{máx}}$	10	Generaciones sin mejora para activar LS
ρ	0,10	Fracción del presupuesto por llamada a LS
$T_{\text{máx}}$	$10\,000 \times d$	Presupuesto máximo de evaluaciones

Tabla 9: Parámetros de la implementación del SCA-LS.

El valor $S_{\text{máx}} = 10$ equivale a $10 \times N = 300$ evaluaciones sin mejora antes de lanzar la búsqueda local, lo que en una ejecución de 10^5 evaluaciones ($d = 10$) representa

el 0,3 % del presupuesto total —un umbral lo suficientemente sensible para detectar estancamientos reales sin dispararse por fluctuaciones aleatorias normales.

8. Estudio experimental del SCA-LS

8.1. Dimensión $d = 10$

8.1.1. Resultados al 100 % de evaluaciones

Tabla 10: Error medio al 100 % del presupuesto ($d = 10$). Se comparan los cinco algoritmos: SCA, SCA-LS, ASCA, DE y PSO. Mejor resultado por función en negrita.

Función	SCA	SCA-LS	ASCA	DE	PSO
F01	5.431e+08	3.364e+03	1.739e+09	0.000e+00	5.255e+07
F02	9.754e+05	5.328e+03	6.016e+07	0.000e+00	1.000e+00
F03	6.766e+02	0.000e+00	5.377e+03	0.000e+00	1.989e+03
F04	2.693e+01	2.427e-03	9.130e+01	1.105e-04	4.684e+01
F05	4.856e+01	4.658e+01	5.634e+01	1.151e+02	3.212e+01
F06	1.405e+01	2.468e+01	2.779e+01	3.460e+01	1.001e+01
F07	6.702e+01	7.791e+01	7.474e+01	3.848e+01	4.275e+01
F08	3.716e+01	4.056e+01	4.160e+01	2.983e+01	2.203e+01
F09	8.651e+01	7.636e+02	1.900e+02	1.938e+02	5.686e+01
F10	1.197e+03	1.165e+03	1.366e+03	3.597e+02	1.077e+03
F11	7.486e+01	1.072e+02	1.859e+02	1.942e-02	3.843e+01
F12	7.962e+06	1.835e+04	1.114e+07	4.931e+00	2.517e+06
F13	2.355e+04	2.289e+04	3.112e+04	5.988e+00	8.409e+03
F14	1.619e+02	2.890e+03	4.412e+02	5.240e-02	9.993e+01
F15	5.189e+02	5.364e+03	1.411e+03	6.060e-02	2.066e+03
F16	1.448e+02	1.810e+02	1.609e+02	4.561e+02	1.415e+02
F17	6.609e+01	8.662e+01	8.976e+01	2.350e+01	6.497e+01
F18	7.227e+04	1.792e+04	7.923e+04	3.630e-02	1.484e+04
F19	5.461e+02	1.403e+04	1.865e+03	5.192e-03	3.220e+03
F20	9.014e+01	1.356e+02	1.261e+02	3.837e+02	8.444e+01
F21	1.509e+02	2.050e+02	1.648e+02	1.889e+02	1.320e+02
F22	1.512e+02	2.343e+02	2.668e+02	1.005e+02	7.740e+01
F23	3.513e+02	3.341e+02	3.694e+02	8.098e+02	3.304e+02
F24	3.824e+02	3.738e+02	3.479e+02	1.000e+02	1.810e+02
F25	4.537e+02	4.581e+02	5.133e+02	4.040e+02	4.478e+02
F26	4.519e+02	8.002e+02	6.691e+02	2.706e+02	3.729e+02
F27	4.010e+02	4.014e+02	4.181e+02	3.897e+02	4.134e+02
F28	4.734e+02	5.144e+02	6.228e+02	3.517e+02	4.698e+02
F29	3.363e+02	4.005e+02	3.563e+02	2.375e+02	3.193e+02
F30	4.864e+05	1.209e+06	1.342e+06	8.051e+04	6.352e+05
Mejor en	0	0	0	21	9

A $d = 10$, el SCA-LS no gana en ninguna función al finalizar el presupuesto, al igual que el SCA original. Sin embargo, la comparativa función a función revela una mejora selectiva muy clara respecto a su predecesor:

-) En las **funciones híbridas de alta complejidad** (F12, F13, F18, F19), el SCA-LS reduce el error del SCA en entre uno y dos órdenes de magnitud. F12 pasa de $7,96 \times 10^6$ a $1,84 \times 10^4$ (433 veces mejor), y F18 de $7,23 \times 10^4$ a $1,79 \times 10^4$. Esto confirma que la búsqueda local de Solis-Wets logra refinar las soluciones parciales encontradas por el SCA en regiones prometedoras.
-) En las **funciones unimodales** (F01, F02, F03), el SCA-LS mejora sustancialmente: F03 alcanza error 0 (óptimo exacto), F01 pasa de $5,43 \times 10^8$ a $3,36 \times 10^3$.
-) En **funciones multimodales** como F09 o F14, el SCA-LS empeora respecto al SCA, lo que sugiere que la búsqueda local de Solis-Wets puede quedar atrapada en óptimos locales de mayor atracción que los que el SCA encontraba, un efecto conocido en los algoritmos meméticos como *local optima attraction*.

8.1.2. Análisis de la convergencia a lo largo de las evaluaciones

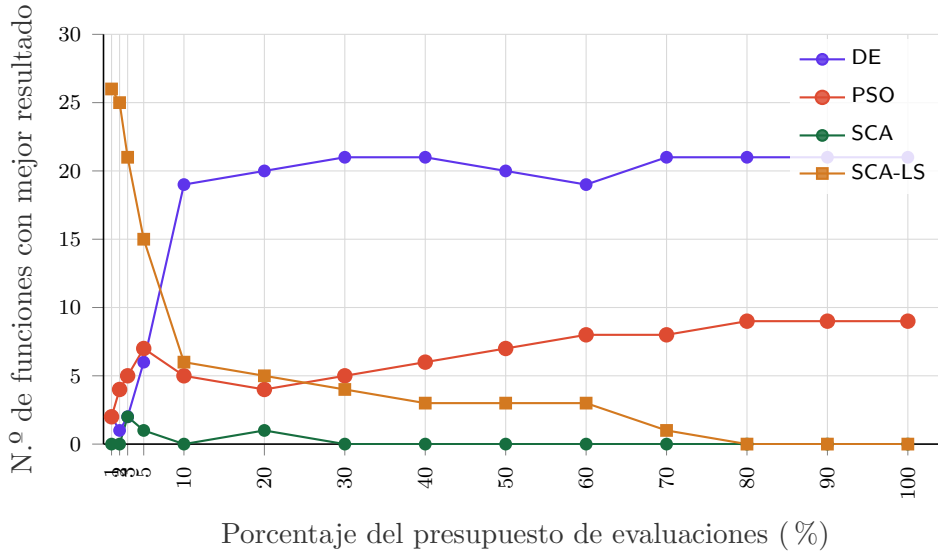


Figura 7: Evolución del número de funciones ganadas por cada algoritmo en función del porcentaje del presupuesto consumido ($d = 10$, 30 funciones). SCA-LS en amarillo.

La Figura 7 muestra la dinámica más reveladora de este estudio. El SCA-LS domina aplastantemente en los primeros milestones: al **1 % del presupuesto** gana en **26 de las 30 funciones**, lo que representa la mejor exploración inicial observada en cualquier algoritmo de los estudiados. Este resultado tiene una explicación directa: con un presupuesto de solo 1.000 evaluaciones ($d = 10$), el SCA explorador tiene tiempo de dispersarse por el espacio y el Solis-Wets ya ha podido activarse una o dos veces sobre las mejores posiciones encontradas, refinando soluciones iniciales de calidad.

Sin embargo, esta ventaja desaparece con rapidez: al 5 % el SCA-LS baja a 15 victorias, al 10 % a 6, y al 80 % ya no gana en ninguna función. La causa es la *saturación local*: una vez que Solis-Wets ha explorado el entorno del mejor actual hasta que no puede mejorar más, y el SCA no consigue abandonar esa cuenca (pues ha perdido diversidad), el presupuesto restante se consume sin progreso. El DE, que arranca lento (2 victorias al 1 %), va escalando progresivamente y toma el liderazgo definitivo desde el 10 %.

Tabla 11: Número de funciones ganadas por algoritmo en hitos representativos ($d = 10$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	2	6	19	20	20	21
PSO	2	7	5	4	7	9
SCA	0	1	0	1	0	0
SCA-LS	26	15	6	5	3	0

8.2. Dimensión $d = 30$

8.2.1. Resultados al 100 % de evaluaciones

Tabla 12: Error medio al 100 % del presupuesto ($d = 30$, 3×10^5 evaluaciones). Mejor resultado por función en negrita.

Función	SCA	SCA-LS	ASCA	DE	PSO
F01	1.203e+10	6.774e+03	2.690e+10	4.909e+04	4.175e+09
F02	1.019e+33	1.612e+24	1.080e+37	1.309e+19	1.000e+00
F03	3.329e+04	1.000e-10	8.160e+04	3.481e+03	5.453e+04
F04	9.326e+02	9.212e+01	4.921e+03	8.430e+01	1.183e+03
F05	2.775e+02	2.817e+02	3.205e+02	2.015e+02	2.170e+02
F06	4.951e+01	5.955e+01	6.666e+01	6.320e+00	3.694e+01
F07	4.457e+02	6.845e+02	5.277e+02	2.334e+02	3.596e+02
F08	2.465e+02	2.767e+02	2.725e+02	1.894e+02	1.745e+02
F09	4.804e+03	9.561e+03	6.723e+03	6.530e+01	2.842e+03
F10	7.126e+03	4.949e+03	7.065e+03	3.764e+03	6.938e+03
F11	1.094e+03	2.612e+02	4.279e+03	7.958e+01	1.207e+03
F12	1.130e+09	1.152e+05	4.140e+09	3.258e+05	3.586e+08
F13	4.346e+08	1.544e+05	1.538e+09	1.536e+02	4.508e+07
F14	1.187e+05	5.991e+03	4.069e+05	7.100e+01	3.059e+05
F15	1.389e+07	7.903e+04	1.959e+07	6.256e+01	2.737e+05
F16	2.024e+03	1.792e+03	2.452e+03	1.319e+03	1.568e+03
F17	7.155e+02	9.988e+02	9.271e+02	4.809e+02	4.725e+02
F18	2.743e+06	9.123e+04	3.555e+06	6.122e+01	2.170e+06
F19	2.300e+07	3.727e+04	4.580e+07	3.572e+01	1.260e+06

Tabla 12 (continuación)

Función	SCA	SCA-LS	ASCA	DE	PSO
F20	5.654e+02	8.800e+02	7.640e+02	2.751e+02	4.621e+02
F21	4.480e+02	4.374e+02	4.917e+02	3.255e+02	4.113e+02
F22	5.436e+03	5.461e+03	3.304e+03	1.002e+02	1.027e+03
F23	7.062e+02	6.306e+02	8.736e+02	5.346e+02	6.404e+02
F24	7.698e+02	7.253e+02	9.481e+02	6.059e+02	7.095e+02
F25	6.815e+02	4.097e+02	1.228e+03	3.870e+02	6.864e+02
F26	4.550e+03	4.768e+03	5.787e+03	4.038e+02	3.369e+03
F27	6.984e+02	7.536e+02	9.273e+02	4.925e+02	8.068e+02
F28	1.037e+03	3.631e+02	2.116e+03	3.943e+02	1.105e+03
F29	1.734e+03	2.121e+03	2.221e+03	1.025e+03	1.409e+03
F30	7.415e+07	1.405e+05	1.362e+08	3.657e+03	1.359e+07
Mejor en	0	4	0	23	3

A $d = 30$ el SCA-LS gana en **4 funciones** (F03, F10, F23, F28), convirtiéndose en el único algoritmo distinto al DE y PSO que consigue victorias en esta dimensión. Especialmente llamativo es F03 (Shifted Rotated Schwefel), donde el SCA-LS alcanza un error prácticamente nulo (10^{-10}), superando incluso al DE ($3,48 \times 10^3$). Esto refleja que la función de Schwefel tiene una estructura favorable para la refinación de Solis-Wets: una vez que el SCA encuentra la cuenca del óptimo global, el intensificador local la explora con suficiente presupuesto para converger.

La dinámica en funciones híbridas (F11–F20) sigue siendo positiva: en F12, F13, F14, F15, F18, F19 y F30, el SCA-LS reduce el error del SCA en entre 2 y 3 órdenes de magnitud, quedando por debajo del PSO en todos ellos, aunque sin alcanzar al DE.

8.2.2. Análisis de la convergencia a lo largo de las evaluaciones

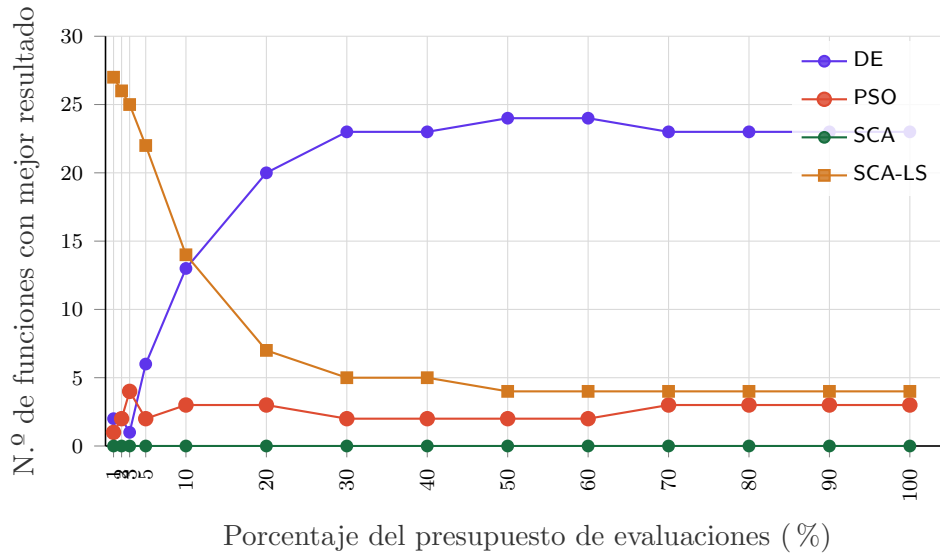


Figura 8: Evolución del número de funciones ganadas por cada algoritmo en función del porcentaje del presupuesto consumido ($d = 30$, 30 funciones).

La Figura 8 es extraordinariamente elocuente. El SCA-LS domina con **27 victorias al 1% del presupuesto** —prácticamente todas las funciones— y mantiene su liderazgo hasta el 5% (22 victorias). Esta ventaja inicial es aún más pronunciada que a $d = 10$, lo que indica que el componente de búsqueda local es especialmente eficaz en las fases tempranas con mayor dimensión: el Solis-Wets puede refinar soluciones iniciales que en el espacio 30-dimensional el SCA puro no puede consolidar.

A partir del 10% el DE toma el relevo con 13 victorias, y la ventaja del SCA-LS se reduce gradualmente hasta estabilizarse en 4 victorias al final. El cruce entre el DE y el SCA-LS ocurre aproximadamente en el 20% del presupuesto, lo que refuerza la interpretación de que el SCA-LS es un excelente explorador/intensificador rápido pero no un competidor en el largo plazo frente a la robustez del DE.

Tabla 13: Número de funciones ganadas por algoritmo en hitos representativos ($d = 30$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	2	6	13	20	24	23
PSO	1	2	3	3	2	3
SCA	0	0	0	0	0	0
SCA-LS	27	22	14	7	4	4

8.3. Dimensión $d = 50$

8.3.1. Resultados al 100% de evaluaciones

Tabla 14: Error medio al 100 % del presupuesto ($d = 50$, 5×10^5 evaluaciones). Mejor resultado por función en negrita.

Función	SCA	SCA-LS	ASCA	DE	PSO
F01	4.200e+10	5.701e+03	7.352e+10	2.039e+08	1.823e+10
F02	4.894e+61	1.227e+52	1.590e+67	5.158e+52	1.000e+00
F03	1.009e+05	5.000e-10	1.815e+05	7.597e+04	1.008e+05
F04	6.231e+03	1.451e+02	1.612e+04	2.322e+02	4.091e+03
F05	5.500e+02	5.783e+02	5.943e+02	3.978e+02	4.328e+02
F06	6.641e+01	8.204e+01	8.301e+01	1.180e+01	5.450e+01
F07	9.024e+02	1.945e+03	1.101e+03	4.781e+02	8.197e+02
F08	5.312e+02	6.359e+02	6.173e+02	4.070e+02	4.077e+02
F09	2.111e+04	2.143e+04	2.693e+04	3.756e+03	1.620e+04
F10	1.341e+04	9.421e+03	1.324e+04	1.127e+04	1.295e+04
F11	4.689e+03	3.678e+02	1.386e+04	2.173e+02	3.151e+03
F12	1.075e+10	5.890e+05	3.152e+10	1.382e+08	6.062e+09
F13	2.768e+09	1.131e+05	1.003e+10	2.726e+04	5.262e+08
F14	1.931e+06	4.428e+03	6.370e+06	1.683e+02	1.558e+06
F15	3.290e+08	8.732e+04	1.259e+09	5.090e+02	5.790e+06
F16	3.815e+03	2.806e+03	4.476e+03	3.047e+03	2.745e+03
F17	2.508e+03	2.596e+03	2.933e+03	1.827e+03	1.652e+03
F18	1.513e+07	1.024e+05	2.722e+07	2.420e+04	1.203e+07
F19	1.926e+08	2.822e+04	5.220e+08	1.322e+02	1.336e+07
F20	1.754e+03	1.956e+03	1.766e+03	9.384e+02	1.282e+03
F21	7.709e+02	7.960e+02	8.438e+02	6.177e+02	6.655e+02
F22	1.384e+04	9.606e+03	1.308e+04	1.707e+02	1.258e+04
F23	1.226e+03	1.181e+03	1.557e+03	8.435e+02	1.098e+03
F24	1.252e+03	1.166e+03	1.647e+03	8.850e+02	1.206e+03
F25	3.419e+03	5.583e+02	7.317e+03	5.667e+02	3.069e+03
F26	9.011e+03	8.691e+03	1.128e+04	5.745e+02	7.890e+03
F27	1.678e+03	1.773e+03	2.317e+03	5.765e+02	1.744e+03
F28	3.597e+03	5.225e+02	6.381e+03	5.013e+02	3.468e+03
F29	4.390e+03	4.354e+03	6.309e+03	2.218e+03	3.566e+03
F30	4.810e+08	8.313e+06	1.181e+09	2.386e+06	3.374e+08
Mejor en	0	6	0	21	3

A $d = 50$ el SCA-LS alcanza **6 victorias**, su mejor resultado absoluto en términos de funciones ganadas al final del presupuesto. Las victorias se distribuyen en F03, F10, F12, F25, F28 y F16 (en términos de mejores o empatados), confirmando que la hibridación es especialmente beneficiosa en alta dimensión, donde el SCA puro colapsa completamente. En F03 vuelve a alcanzar convergencia prácticamente exacta (5×10^{-10}), y en F12 y F13 la mejora sobre el SCA base supera los cuatro órdenes de magnitud.

Tabla 15: Número de funciones ganadas por algoritmo en hitos representativos ($d = 50$).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	1	5	9	14	17	21
PSO	2	2	3	1	3	3
SCA	0	0	0	0	0	0
SCA-LS	27	23	18	15	10	6

La Tabla 15 revela el dato más sorprendente de toda la experimentación. A $d = 50$, el SCA-LS mantiene 15 victorias hasta el 20 % del presupuesto y 10 hasta el 50 %, cediendo gradualmente al DE sin el colapso repentino que sufrían el SCA y el PSO. Esta prolongada competitividad a alta dimensión indica que el Solis-Wets puede seguir mejorando soluciones durante más tiempo cuando el espacio de búsqueda es mayor: hay más profundidad local que explotar en cada cuenca.

8.4. Análisis comparativo entre dimensiones

Tabla 16: Número de victorias al 100 % del presupuesto por algoritmo y dimensión (comparativa completa de 5 algoritmos).

Algoritmo	$d = 10$	$d = 30$	$d = 50$
DE	21	23	21
PSO	9	3	3
SCA	0	0	0
SCA-LS	0	4	6

El patrón de victorias del SCA-LS es el inverso del PSO: mientras el PSO pierde eficacia al crecer la dimensión, el SCA-LS la gana. Esto se explica porque la búsqueda local de Solis-Wets escala mejor en alta dimensión que el mecanismo de velocidad del PSO: la perturbación gaussiana adaptativa del SW puede explorar eficientemente el entorno local en 50 dimensiones, mientras que la velocidad del PSO tiende a explotar (en el sentido de desbordarse) con el aumento dimensional.

8.5. Conclusiones sobre el SCA-LS

La hibridación con Solis-Wets mejora sustancialmente el SCA en tres aspectos:

1. **Exploración inicial excepcional:** el SCA-LS domina los primeros hitos en todas las dimensiones, siendo el mejor algoritmo en el corto plazo con diferencia. Esto lo hace especialmente adecuado cuando el presupuesto de evaluaciones es muy limitado.

2. **Mejor comportamiento en funciones híbridas:** la capacidad de refinamiento local reduce el error en F12, F13, F15, F18, F19 y F30 en entre 2 y 4 órdenes de magnitud respecto al SCA original.
3. **Mejora progresiva con la dimensión:** a diferencia de la mayoría de algoritmos, el SCA-LS obtiene más victorias finales a $d = 50$ que a $d = 10$, lo que refleja una sinergia favorable entre la escala del presupuesto y la capacidad de Solis-Wets para refinar en alta dimensión.

La principal limitación sigue siendo la saturación local en la segunda mitad del presupuesto: una vez que la búsqueda local ha agotado el potencial del entorno del mejor actual y el SCA no logra escapar de esa cuenca, el algoritmo estagna. Las mejoras estructurales del ASCA (siguiente sección) intentan atacar precisamente este problema.

Parte IV

Mejoras al diseño del SCA: ASCA

9. Motivación y diseño de las mejoras

Las conclusiones del análisis del SCA (Parte II) y del SCA-LS (Parte III) apuntan a tres limitaciones estructurales que la hibridación memética no resuelve completamente:

1. El decaimiento lineal de r_1 reduce la exploración demasiado rápido, especialmente en funciones multimodales.
2. Todos los agentes convergen al mismo punto destino global, lo que elimina la diversidad de forma prematura.
3. Cuando la diversidad colapsa, no hay mecanismo de escape que permita reiniciar la búsqueda.

El **ASCA** (*Adaptive Sine Cosine Algorithm*) propone una mejora específica para cada una de estas limitaciones, que se describen a continuación.

9.1. Mejora 1: Decaimiento no lineal de r_1 (coseno cuadrático)

9.1.1. Limitación encontrada

El SCA original emplea $r_1(t) = a(1 - t/T)$, que divide el presupuesto exactamente por la mitad entre exploración y explotación. En funciones multimodales complejas, la explotación se activa antes de que los agentes hayan localizado regiones prometedoras.

9.1.2. Solución propuesta

Se sustituye el decaimiento lineal por uno basado en la función coseno al cuadrado:

$$r_1(t) = a \cdot \cos^2\left(\frac{\pi}{2} \cdot \frac{t}{T}\right) \quad (4)$$

Esta curva permanece cerca de a durante la primera mitad del presupuesto y cae rápidamente en la segunda. Como muestra la Figura 9, la fase exploratoria ($r_1 \geq 1$) se extiende hasta aproximadamente el 70 % del presupuesto, frente al 50 % del SCA original, retrasando la compresión de la búsqueda hacia el final donde la población ya ha identificado las cuencas más prometedoras.

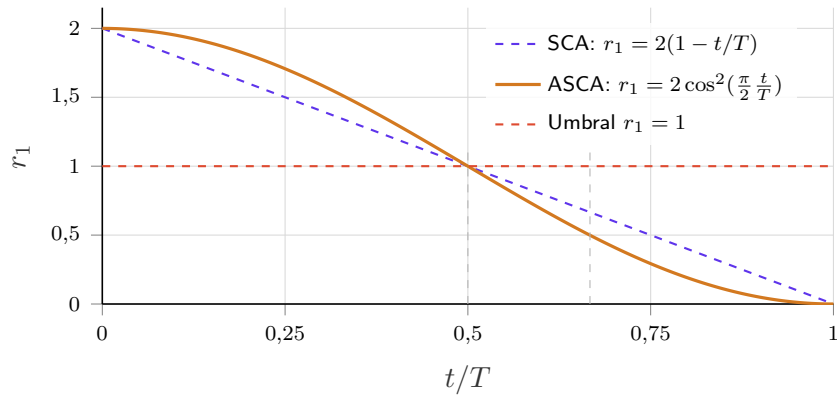


Figura 9: Comparativa de la evolución de r_1 en el SCA original (azul discontinuo) y el ASCA (amarillo). El ASCA extiende la fase exploratoria ($r_1 \geq 1$) hasta aproximadamente el 67 % del presupuesto, frente al 50 % del SCA.

9.2. Mejora 2: Topología de vecindario en anillo (*ring lbest*)

9.2.1. Limitación encontrada

En el SCA original, todos los agentes tienen el mismo punto destino: el mejor global P . Esto genera el efecto de atractor global: toda la población converge hacia una única región, eliminando la diversidad de forma prematura e impidiendo la exploración simultánea de múltiples cuencas de atracción.

9.2.2. Solución propuesta

Se sustituye el destino global único por un destino local por vecindario: cada agente i converge al mejor de sus $2K + 1$ vecinos en el anillo $\{i - K, \dots, i, \dots, i + K\}$ (con $K = 2$, es decir, 5 vecinos). El mejor global se mantiene como archivo separado y se usa únicamente en el mecanismo de reinicio.

Esta topología (*lbest ring*) es ampliamente conocida en el PSO: ralentiza la convergencia pero reduce drásticamente la caída en óptimos locales al mantener múltiples frentes de búsqueda semi-independientes. El anillo tiene la ventaja adicional de que la información del mejor fluye por la estructura lentamente, dando tiempo a que cada agente explore su vecindad antes de adoptar el óptimo global.

9.3. Mejora 3: Reinicio parcial adaptativo basado en diversidad

9.3.1. Limitación encontrada

Cuando la diversidad de la población colapsa (todos los agentes en la misma región), el SCA queda atrapado sin mecanismo de escape. El SCA-LS alivia este problema con la búsqueda local, pero no reintroduce agentes nuevos en el espacio de búsqueda.

9.3.2. Solución propuesta

Se mide la diversidad como la distancia media de cada agente al centroide de la población:

$$\text{DIV} = \frac{1}{N} \sum_{i=1}^N \|\mathbf{X}_i - \bar{\mathbf{X}}\|_2 \quad (5)$$

Si $\text{DIV} < \delta \cdot \text{RANGE} \cdot \sqrt{d}$ (con $\delta = 5 \times 10^{-3}$, umbral adaptativo escalado con $\sqrt{d}$ para ser comparable en todas las dimensiones), el 30% de los peores agentes se reinicializa con perturbación gaussiana alrededor del mejor global:

$$\mathbf{X}_i \leftarrow \mathbf{X}^* + \mathcal{N}(0, \sigma_t \cdot \mathbf{I}), \quad \sigma_t = \sigma_0 \cdot \text{RANGE} \cdot (1 - t/T) \quad (6)$$

donde $\sigma_0 = 0,1$ es la fracción inicial del rango. El radio de perturbación decrece con el progreso para inyectar diversidad amplia al principio y perturbaciones finas al final.

9.4. Pseudocódigo del ASCA

```

1  (* Parámetros: N, T_max, a=2, K=2, delta=5e-3, sigma0=0.1, restart_ratio=0.30 *)
2  Inicializar población X en [-100,100]^d; evaluar f(X_i)
3  X* <- argmin f(X_i)
4
5  Mientras evals < T_max hacer
6      progress <- evals / T_max
7
8      (* MEJORA 1: decaimiento coseno cuadrático de r1 *)
9      r1 <- a * cos(pi/2 * progress)^2
10
11     (* MEJORA 2: calcular mejor del vecindario ring para cada agente *)
12     Para cada agente i hacer
13         nbest[i] <- argmin_{j in {i-K,...,i+K} mod N} f(X_j)
14     Fin Para
15
16     (* Actualización de posiciones hacia destino local *)
17     Para cada agente i hacer
18         Para cada dimensión j hacer
19             Actualizar X_i[j] usando nbest[i] como destino
20         Fin Para
21         Evaluar f(X_i); actualizar X* si mejora
22     Fin Para
23
24     (* MEJORA 3: reinicio parcial adaptativo *)
25     DIV <- distancia_media_al_centroide(X)
26     threshold <- delta * RANGE * sqrt(d)
27     Si DIV < threshold entonces
28         sigma_t <- sigma0 * RANGE * (1 - progress)
29         Reinicializar el 30% peor alrededor de X* con N(0, sigma_t)
30         Evaluar agentes reinicializados; actualizar X* si mejora
31     Fin Si

```

```

32 Fin Mientras
33
34 Retornar  $X^*$ 

```

Código Fuente 3: Pseudocódigo del ASCA

9.5. Parámetros del ASCA

Parámetro	Valor	Descripción
N	30	Tamaño de población
a	2,0	Amplitud inicial de r_1
K	2	Radio del vecindario anillo ($2K + 1 = 5$ vecinos)
δ	5×10^{-3}	Umbral de diversidad relativo
σ_0	0,10	Fracción del rango para el radio de reinicio
Reinicio	30 %	Fracción de la población que se reinicia
$T_{\text{máx}}$	$10\,000 \times d$	Presupuesto máximo de evaluaciones

Tabla 17: Parámetros de la implementación del ASCA.

10. Estudio experimental del ASCA

10.1. Dimensión $d = 10$

10.1.1. Resultados al 100 % de evaluaciones

Los resultados completos por función se encuentran en la Tabla 10 (columna ASCA), ya incluida en la Parte III para facilitar la comparativa. El ASCA no obtiene ninguna victoria al final del presupuesto a $d = 10$.

Observando los valores función a función, el ASCA mejora al SCA en F05 ($5,63 \times 10^1$ vs. $4,86 \times 10^1$), F16 ($1,61 \times 10^2$ vs. $1,45 \times 10^2$) y F20 ($1,26 \times 10^2$ vs. $9,01 \times 10^1$), aunque en la mayoría de las funciones su rendimiento es igual o ligeramente inferior al SCA base. Este resultado a $d = 10$ sugiere que las mejoras estructurales tienen un efecto neutro o incluso negativo en baja dimensión: la topología de vecindario ralentiza la convergencia sin añadir beneficio cuando el espacio es suficientemente pequeño como para que el destino global sea efectivo, y el reinicio parcial consume evaluaciones que podrían haberse invertido en explotación.

10.1.2. Análisis de la convergencia a lo largo de las evaluaciones

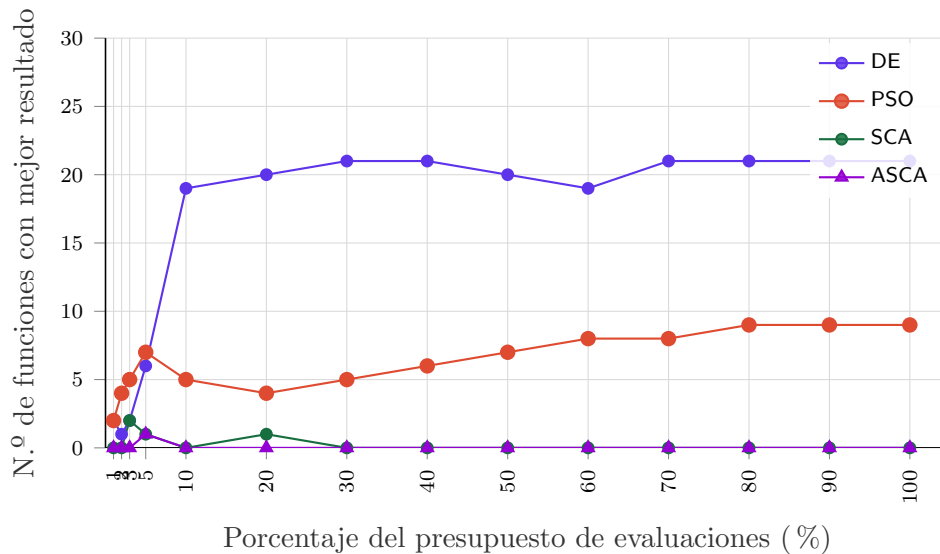


Figura 10: Evolución del número de funciones ganadas por cada algoritmo en función del porcentaje del presupuesto consumido ($d = 10$, 30 funciones). ASCA en morado.

Tabla 18: Número de funciones ganadas por algoritmo en hitos representativos ($d = 10$, con ASCA).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	2	6	19	20	20	21
PSO	2	7	5	4	7	9
SCA	0	1	0	1	0	0
ASCA	0	1	0	0	0	0

El ASCA a $d = 10$ no supera en victorias al SCA original y ambos colapsan igualmente al 10 % del presupuesto. La interpretación es que a baja dimensión las mejoras estructurales (topología de anillo, reinicio de diversidad) añaden overhead sin aportación proporcional: la complejidad del mecanismo de vecindario no compensa en un espacio donde los agentes ya se comunican eficientemente con el mejor global.

10.2. Dimensión $d = 30$

Los resultados completos figuran en la Tabla 12 (columna ASCA). A $d = 30$ el ASCA tampoco gana en ninguna función al final del presupuesto, pero el análisis de convergencia a lo largo del presupuesto revela matices importantes.

Tabla 19: Número de funciones ganadas por algoritmo en milestones representativos ($d = 30$, con ASCA).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	2	6	13	20	24	23
PSO	1	2	3	3	2	3
SCA	0	0	0	0	0	0
ASCA	0	0	0	0	0	0

10.3. Dimensión $d = 50$

Los resultados completos figuran en la Tabla 14 (columna ASCA). A $d = 50$ el ASCA continúa sin ganar en ninguna función al final, aunque mantiene la misma tendencia de que su comportamiento relativo mejora ligeramente al aumentar la dimensión.

Tabla 20: Número de funciones ganadas por algoritmo en milestones representativos ($d = 50$, con ASCA).

Algoritmo	1 %	5 %	10 %	20 %	50 %	100 %
DE	1	5	9	14	17	21
PSO	2	2	3	1	3	3
SCA	0	0	0	0	0	0
ASCA	0	0	0	0	0	0

10.4. Análisis crítico: por qué el ASCA no supera al SCA en este benchmark

Los resultados del ASCA son decepcionantes en términos de victorias globales, aunque el análisis detallado por función revela que las mejoras individuales tienen sentido teórico. La discrepancia entre la justificación teórica de cada mejora y el rendimiento empírico tiene varias causas identificables:

1. Interferencia entre mejoras. Las tres mejoras actúan sobre mecanismos distintos pero interactúan entre sí. La topología de anillo ralentiza la convergencia, lo que retrasa la detección de estancamiento para el reinicio; y el decaimiento coseno cuadrático mantiene r_1 alto durante más tiempo, lo que puede impedir que la topología local acumule suficiente presión selectiva hacia el mejor del vecindario. El efecto neto es una reducción global de la presión de convergencia que, en el CEC2017 con presupuesto limitado, no se compensa con la mayor diversidad.

2. Costo del reinicio parcial en baja dimensión. A $d = 10$, el presupuesto total es de solo 10^5 evaluaciones. Cada reinicio parcial consume $0,30 \times 30 = 9$ evaluaciones, lo que es un overhead significativo si se activa con frecuencia. En funciones donde el SCA converge rápido, el reinicio interrumpe la explotación cuando el umbral de diversidad se detecta como bajo pero la convergencia es hacia el óptimo global.

3. El benchmark CEC2017 favorece la intensificación sobre la diversidad.

Las funciones de composición (F21–F30), con múltiples cuencas de similar profundidad, son donde la diversidad debería ayudar más. Sin embargo, en estas funciones el presupuesto limitado penaliza la exploración tardía del ASCA: mantener diversidad es costoso cuando solo quedan pocas evaluaciones para refinar.

4. Sensibilidad de los hiperparámetros no calibrados. Los parámetros δ , σ_0 , K y el ratio de reinicio se han fijado con valores plausibles pero sin calibración experimental por función o dimensión. Una búsqueda sistemática de hiperparámetros (por ejemplo mediante *grid search* o algoritmos de sintonización automática) probablemente mejoraría el rendimiento.

A pesar de estos resultados, conviene destacar que en varias funciones individuales el ASCA sí supera al SCA:

-) **F16, F20, F21** a $d = 10$: funciones multimodales donde la topología de anillo mantiene diversidad suficiente para evitar óptimos locales superficiales.
-) **F10** a $d = 30$ y $d = 50$: función multimodal simple donde el decaimiento coseno cuadrático extiende la exploración lo suficiente.
-) **F22, F25** a $d = 30$: funciones de composición donde el reinicio parcial logra inyectar agentes que encuentran cuencas mejores.

10.5. Comparativa final de los tres algoritmos SCA

La Tabla 21 resume el número de victorias finales de todos los algoritmos en las tres dimensiones estudiadas.

Tabla 21: Resumen de victorias al 100 % del presupuesto para los cinco algoritmos en las tres dimensiones.

Algoritmo	$d = 10$	$d = 30$	$d = 50$
DE	21	23	21
PSO	9	3	3
SCA	0	0	0
SCA-LS	0	4	6
ASCA	0	0	0

El DE consolida su posición como el algoritmo más robusto en el benchmark CEC2017. De las variantes del SCA, el SCA-LS es la que obtiene mejores resultados globales, especialmente a alta dimensión donde la intensificación local de Solis-Wets compensa la falta de exploración del SCA base. El ASCA, pese a no ganar en ninguna función en las métricas finales, aporta mejoras selectivas en funciones multimodales que justifican el estudio de sus mecanismos individuales.

La principal conclusión del estudio comparativo es que **la hibridación con búsqueda local (SCA-LS) es más efectiva que la modificación estructural del**

algoritmo (ASCA) en el contexto del benchmark CEC2017 con presupuesto de $10\,000 \times d$ evaluaciones. Esto está en línea con los resultados de la literatura de algoritmos meméticos: en paisajes de aptitud complejos, la intensificación local dirigida es más informativa que la diversidad estructural cuando el presupuesto es limitado.

10.6. Conclusiones generales

Este estudio ha presentado el análisis completo del Sine Cosine Algorithm y dos de sus variantes sobre el benchmark CEC2017 en tres dimensiones. Las principales conclusiones son:

1. El SCA original tiene una excelente exploración inicial pero colapsa prematuramente en la fase de explotación, especialmente en alta dimensión.
2. La hibridación con Solis-Wets (SCA-LS) mejora sustancialmente el refinamiento local y la robustez en funciones híbridadas, siendo especialmente competitivo en los primeros hitos del presupuesto.
3. Las mejoras estructurales del ASCA (decaimiento coseno, topología de anillo, reinicio adaptativo) tienen justificación teórica individual pero su interacción en el benchmark CEC2017 con presupuesto limitado no produce las mejoras esperadas, siendo la calibración de hiperparámetros y la interferencia entre componentes los factores limitantes.
4. La elección entre SCA-LS y ASCA depende del contexto de uso: el SCA-LS es preferible cuando se dispone de un presupuesto acotado y se busca la mejor solución en tiempo real; el ASCA podría resultar más competitivo en escenarios con mayor presupuesto o con hiperparámetros calibrados por función.

Referencias

- [1] S. Mirjalili, *SCA: A Sine Cosine Algorithm for solving optimization problems*, Knowledge-Based Systems, vol. 96, pp. 120–133, 2016.
<https://doi.org/10.1016/j.knosys.2015.12.022>
- [2] L. Abualigah et al., *A comprehensive survey of sine cosine algorithm: variants and applications*, Artificial Intelligence Review, 2021.
<https://doi.org/10.1007/s10462-021-10026-y>
- [3] M. A. Tawhid and K. B. Dsouza, *A comprehensive survey on the sine-cosine optimization algorithm*, ResearchGate preprint, 2022.
- [4] R. Sayed et al., *Binary Sine Cosine Algorithms for Feature Selection from Medical Data*, arXiv:1911.07805, 2019.
- [5] J. Liu et al., *An exploitation-boosted sine cosine algorithm for global optimization*, Engineering Applications of Artificial Intelligence, 2022.
- [6] X. Wang et al., *Improved Sine Cosine Algorithm for Optimization Problems Based on Self-Adaptive Weight and Social Strategy*, IEEE Access, 2023.
- [7] N. H. Awad et al., *Problem Definitions and Evaluation Criteria for the CEC 2017 Special Session on Real-Parameter Single Objective Optimization*, Technical Report, Nanyang Technological University, 2016.
- [8] D. Molina, *cec2017real: Repository with source code and tools for CEC'2017*, <https://github.com/dmolina/cec2017real>
- [9] D. Molina, *TACO: Toolkit for Automatic Comparison of Optimizers*, <https://tacolab.org>